

Dosimeter Firmware

iThemba LABS beam-loss monitor

iThemba LABS Test Rig — FirmwareUpdate/ILT_OS/Applications/Dosimeter

September 11, 2026

Abstract

The dosimeter counts pulses on six independent channels and publishes a reading each period. This document describes the firmware that does it: how it is layered, how the sampling period is kept honest, the network interface it presents, how it is configured and built, and where it departs from the original `ithemba_control_system/dosimeter` firmware it replaces.

Contents

1	What the firmware does	2
2	How the code is arranged	2
2.1	Files	2
3	Threads	3
3.1	The period is kept honest	3
4	The sample	3
4.1	Period limits	4
5	Network interface	4
5.1	HTTP routes	4
5.2	Reading and setting	4
5.3	The reading stream	4
6	Configuration	5
6.1	Backwards compatibility	5
7	Building and flashing	6
7.1	Host build	6
7.2	Cross build	6
7.3	Flashing	6
8	Hardware interface	6
9	Indicators	7
10	Known limitations	7

1 What the firmware does

Six pulse-shaper channels feed six MCU timers configured as external-clock counters. Once per period the firmware reads all six counters, resets them, and publishes the six counts together with a sequence number and a timestamp.

The counting itself is done in hardware. The firmware never sees an individual pulse and there is no per-pulse interrupt, so the count is unaffected by how busy the CPU is — which is what makes the reading trustworthy at high rates.

2 How the code is arranged

The firmware is three layers, and the rule that keeps them apart is what lets the same application run on more than one board:

Rule Nothing under `ILT_OS/Lib` or `ILT_OS/Applications` may include a chip header. Everything an application knows about the hardware arrives through `BSP/Include/Bsp/*.h`.

Layer	Responsibility
<code>BSP/Boards/<board></code>	The chip. HAL initialisation, pin mapping, the counter timers, Ethernet. Selected with <code>-DILT_BOARD</code> .
<code>ILT_OS/Lib</code>	Portable services over FreeRTOS and lwIP: Thread, Mutex, Queue, Timer, NetworkStack, HttpServer, StreamServer.
<code>ILT_OS/Applications/Dosimeter</code>	This firmware. The sampling loop and the network interface.

Exactly one application is linked into an image, chosen with `-DILT_APP`, and reached through one entry point:

```
extern "C" void ILT_ApplicationStart(void)
{
    bsp::boardInit();
    g_supervisor.start();
}
```

2.1 Files

File	Contains
<code>DosimeterApp.cpp</code>	Supervisor thread, network bring-up, HTTP routes, the stream listener.
<code>Dosimeter.h/.cpp</code>	The sampling thread, the period, and the latest sample.
<code>IndexPage.h</code>	The built-in page, a string literal in flash.
<code>CMakeLists.txt</code>	Configuration options and the <code>ilt_app</code> target.

`Dosimeter.h/.cpp` are shared verbatim with the combined `ILT_TESTRIG` image, so behaviour cannot drift between the two.

3 Threads

Two threads run, plus lwIP's own `tcpip_thread`.

Thread	Priority	Does
<code>dosimeter-sup</code>	Normal	Brings the network up, registers routes, starts the stream, then every 250 ms updates the LEDs.
<code>dosimeter</code>	Normal	Sleeps until the next deadline, reads and resets the six counters, publishes.

The sampling thread is started only once the network is up, so a record can never be produced before there is somewhere to send it.

3.1 The period is kept honest

The sampling loop waits on an *absolute* deadline rather than sleeping for the period:

```
std::uint32_t next = ilt::kernel::tickCount();

for (;;)
{
    next += periodS_ * 1000U;
    ilt::kernel::delayUntil(next);
    /* read, format, publish */
}
```

Listing 1: From `Dosimeter.cpp`.

Had it slept for the period instead, the time spent reading, formatting and handing the record to the network would be added to every cycle and the period would drift steadily long. Measured on hardware, consecutive records at a two second period carried timestamps 14021, 16021 and 18021 ms — exactly 2000 ms apart.

4 The sample

```
struct Sample
{
    std::uint32_t sequence; /* increments once per sample */
    std::uint32_t uptimeMs; /* when it was taken */
    std::uint32_t counts[6]; /* pulses this period */
};
```

Counts are *per period*, not cumulative: the counters are read and reset together, so each record reports what arrived since the previous record.

The latest sample is kept under a mutex so that an HTTP request, which is served from lwIP's thread, cannot read a half-written set of six counts while the sampling thread is storing them.

Constant	Value	Meaning
kMinPeriodS	1 s	Shortest accepted period.
kMaxPeriodS	3600 s	Longest accepted period, one hour.

4.1 Period limits

A period outside the range is rejected and the existing period is kept.

5 Network interface

The original firmware owned a TCP server for commands and a TCP client that dialled out with readings. This firmware splits that by shape:

- **requests** — HTTP on port 80, one path per operation, JSON replies;
- **readings** — one TCP port that clients connect to and stay connected to, receiving a JSON object per line as each period elapses.

5.1 HTTP routes

Route	Returns
/index.html	The built-in page.
/api/status.json	Board, address, uptime, link, stream counters.
/api/dosimeter.json	The latest sample.
/api/dosimeter/period.json	Sets the period. Takes <code>?seconds=1..3600</code> .
/favicon.ico	204 No Content.

5.2 Reading and setting

```
$ curl http://192.168.7.105/api/dosimeter.json
{"present":false,"seq":13,"t_ms":13021,"period_s":1,"ch":[0,0,0,0,0,0]}

$ curl 'http://192.168.7.105/api/dosimeter/period.json?seconds=2'
{"ok":true,"period_s":2,"min":1,"max":3600}

$ curl 'http://192.168.7.105/api/dosimeter/period.json?seconds=9999'
{"ok":false,"period_s":2,"min":1,"max":3600}
```

A rejected period leaves the current one in place and the reply says so, along with the bounds — so a caller that guessed wrong learns the range without consulting this document.

`present` is false when the board has no pulse counters, which is how a NUCLEO development board reports itself.

5.3 The reading stream

One JSON object per line, one per period:

```
{"type":"dosimeter","seq":14,"t_ms":14021,"period_s":2,"ch":[0,0,0,0,0,0]}
{"type":"dosimeter","seq":15,"t_ms":16021,"period_s":2,"ch":[0,0,0,0,0,0]}
```

```
{"type":"dosimeter","seq":16,"t_ms":18021,"period_s":2,"ch":[0,0,0,0,0,0]}
```

A client that connects late receives the next record within one period, so no special case is needed for a snapshot on connect.

Note `status.json` reports dropped alongside `sent`. A record is dropped when the pool of in-flight records to lwIP is full — a slow or stalled client, not a lost count. The counts themselves are never lost; only the publication of them is.

6 Configuration

```
cmake --preset Debug -DILT_APP=Dosimeter -DDOSIMETER_STREAM_PORT=1973
```

Variable	Default
DOSIMETER_STACK_BYTES	1536
DOSIMETER_HOSTNAME	ilt-dosimeter
DOSIMETER_STREAM_PORT	1973
DOSIMETER_USE_DHCP	ON
DOSIMETER_DHCP_TIMEOUT_MS	15000
DOSIMETER_FALLBACK_IP	192.168.7.105
DOSIMETER_FALLBACK_NETMASK	255.255.255.0
DOSIMETER_FALLBACK_GATEWAY	192.168.7.1

Table 1: Configuration, with the values that preserve the original firmware’s addressing.

6.1 Backwards compatibility

The defaults are taken from `ithemba_control_system/dosimeter`:

Original	Value	Here
LOCAL_IP	192.168.7.105	fallback address
SERVER_SEND_PORT	1973	stream port
SERVER_RECV_PORT	1972	<i>not preserved</i>
SERVER_IP	192.168.7.1	gateway
addressing	<code>dhcp_start()</code>	DHCP on

DHCP is on because the original called `dhcp_start()`; the declared `LOCAL_IP` of .105 is used as the fallback, on the assumption that is what the lease was reserved as. Verified on hardware: with no DHCP server present the firmware waits the timeout and settles on 192.168.7.105.

Caution The command port is **not** backwards compatible. The original listened on 1972 and spoke a binary framing with a `0xA5` sync byte; requests here are HTTP on port 80. Only the addressing and the data port are preserved.

7 Building and flashing

7.1 Host build

```
cd FirmwareUpdate
cmake -S . -B build/HostDos -G Ninja \
      -DILT_BOARD=Host -DILT_APP=Dosimeter
cmake --build build/HostDos
./build/HostDos/FirmwareUpdate
```

7.2 Cross build

```
cmake -S . -B build/F767_Dosimeter -G Ninja -DCMAKE_BUILD_TYPE=Debug \
      -DCMAKE_TOOLCHAIN_FILE=cmake/arm-none-eabi-gcc.cmake \
      -DILT_BOARD=NucleoF767ZI -DILT_APP=Dosimeter
cmake --build build/F767_Dosimeter
```

Measured footprint on the F767: 98.8 kB of flash and 97.1 kB of RAM.

7.3 Flashing

```
STM32_Programmer_CLI -c port=SWD mode=UR \
  -w build/F767_Dosimeter/FirmwareUpdate.hex -v -rst
```

The console is USART3 on the ST-LINK virtual COM port at 115 200 baud. A healthy boot reads:

```
Dosimeter on NUCLEO-F767ZI
rig: no pulse counters on this board
http: listening on port 80
stream: listening on port 1973
net: link up, requesting an address
net: ready -- http://192.168.7.105/ and stream on port 1973
```

8 Hardware interface

Everything the application knows about the counters comes through `Bsp/PulseCounter.h`:

```
inline constexpr std::size_t kChannelCount = 6U;

bool      available() noexcept;
void      init() noexcept;
std::uint32_t read(std::size_t channel) noexcept;
void      reset(std::size_t channel) noexcept;
void      readAndReset(std::uint32_t *counts) noexcept;
```

`readAndReset()` exists rather than six `read()/reset()` pairs so that all six channels are sampled against as nearly the same instant as the hardware allows; reading them one at a time would smear the six counts across the time it takes to service them.

9 Indicators

LED	Meaning
Status	Solid once addressed; blinking while it is not.
Activity	Follows records leaving the stream.
Fault	On when the board reports no pulse counters.

10 Known limitations

1. **The command protocol is not backwards compatible.** See §6.1.
2. **The HTTP port is fixed at 80.** It is lwIP's `httpd`, set board-wide in `lwipopts.h` rather than per application.
3. **The custom board's counter timers are not final.** Merging the two rigs puts the degrader's stepper PWM and the dosimeter's gating timebase both on TIM12. The board README records the conflict; the gating timebase has to move to a spare timer before the dosimeter runs on that board.